Министерство образования Республики Беларусь

Учреждение образования

«Брестский государственный технический университет»

Кафедра ИИТ

Лабораторная работа №4

По дисциплине «Обработка изображений в ИС»

Тема: «Трекинг множественных объектов»

Выполнил:

Студент 4 курса

Группы ИИ-26

Пилипук М.А.

Проверил:

Андренко К.В.

Брест 2026

Цель работы: исследовать применение алгоритмов трекинга на базе обученной сети-детектора объектов.

Задание: Используя сеть из ЛР3, реализовать логику для отслеживания множественных объектов, используя Ultralytics YOLO. Применить алгоритмы BoT-Sort и ByteTrack. Исследовать влияние изменения параметров в конф.файлах на качество трекинга. В качестве исходных видеоматериалов для экспериментов использовать видео-ролики из сети.

В качестве исходного материала был выбран клип группы Linkin Park — One Step Closer, ибо трекинг водяных счетчиков как-то скучно.

Часть 1:

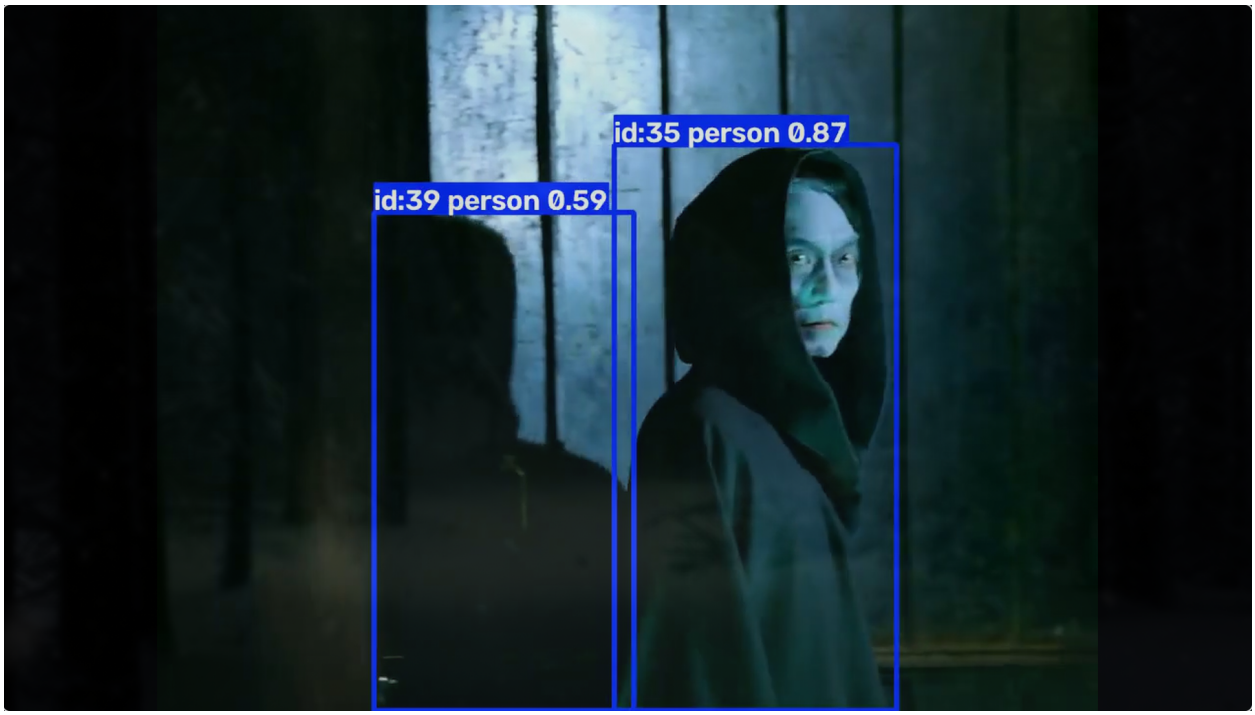
Первый код:

```
from ultralytics import YOLO


model = YOLO("yolo12m.pt")

results = model.track(
source="/content/drive/MyDrive/SHARED/One Step Closer.mp4",
tracker="bytetrack.yaml",
classes=[0],
save=True,
)
```

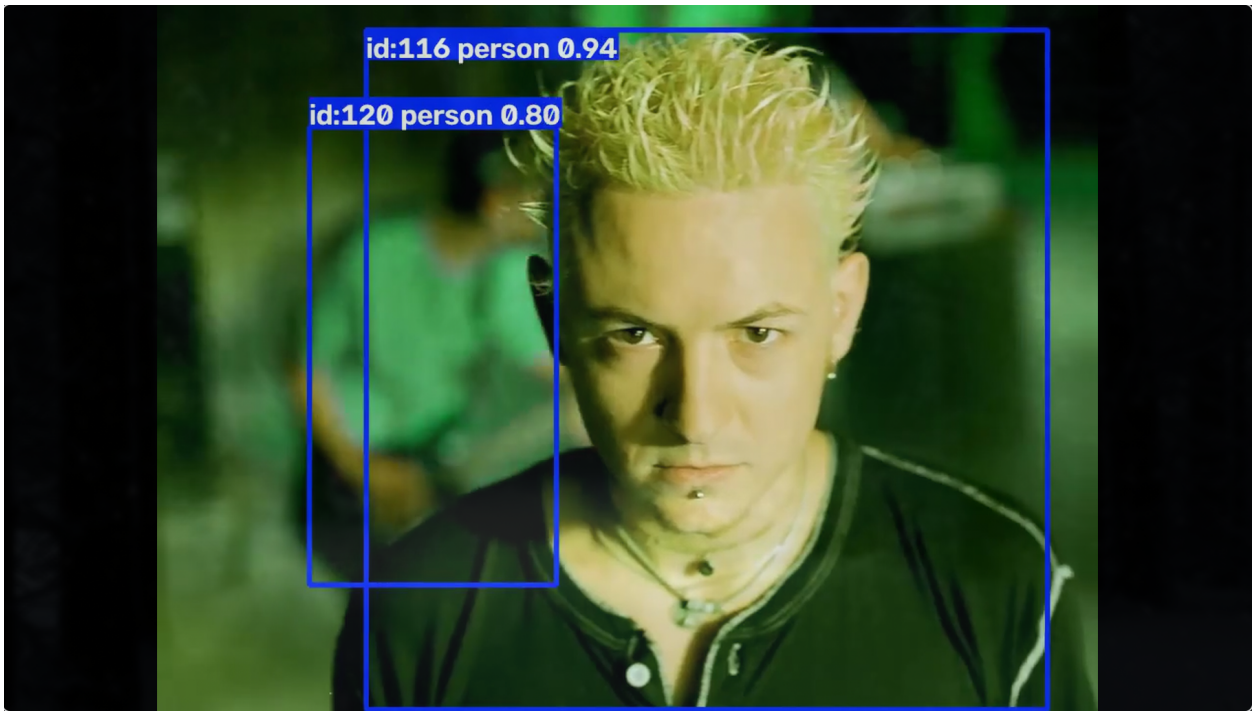
Итоговые видео доступны по ссылке - https://drive.google.com/drive/folders/1MRTzvEsRHKox2DeYJzAtLKeLwRtDbtSA?usp=sharing

Визуальные результаты:

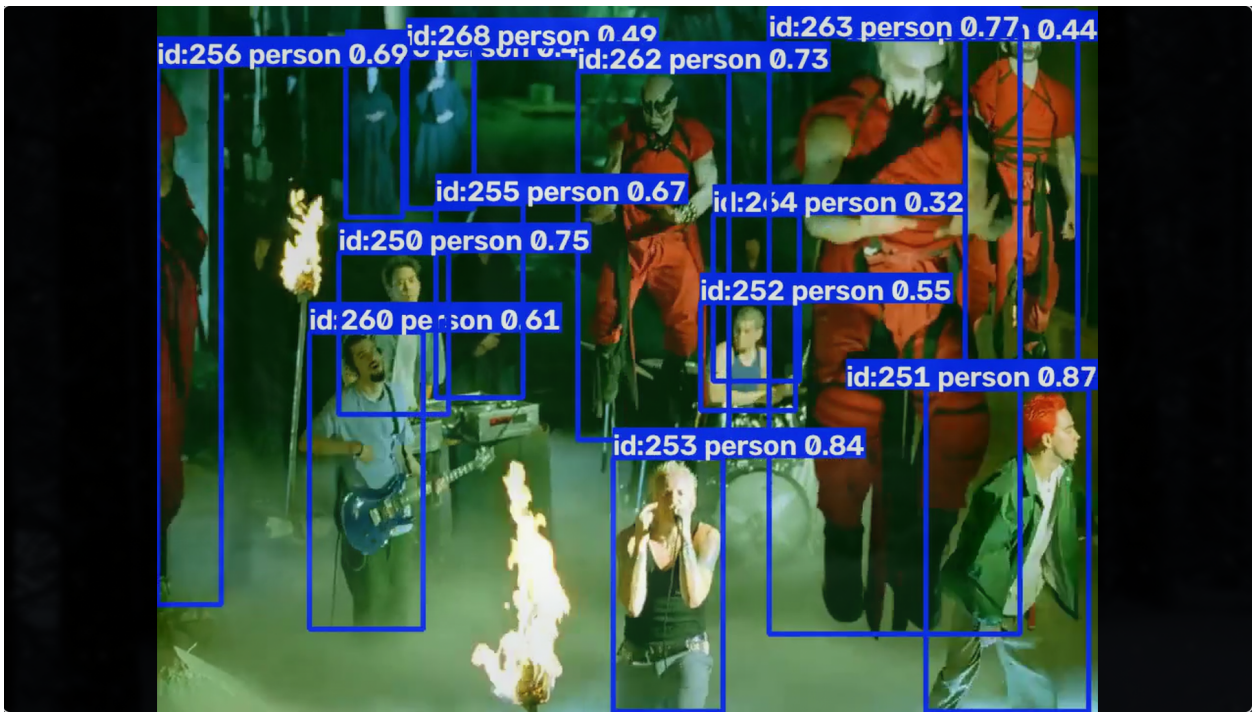
Люди прекрасно детектятся!



Первая интересная проблемка — тень детектится как человек. Отсюда можно сделать вывод, что модель детектит человека больше по силуэту, нежели по лицу. Это одновременно и хорошо, ибо модель может засечь человека в малоосвещенном помещении, и плохо — так как может привести к таким «аномалиям».



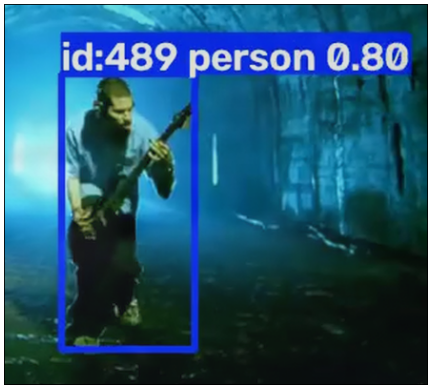
Какой же Честер красавчик\милашка. Хочу стать как он, крутым.

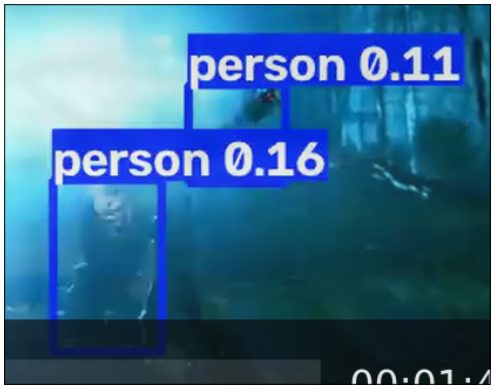


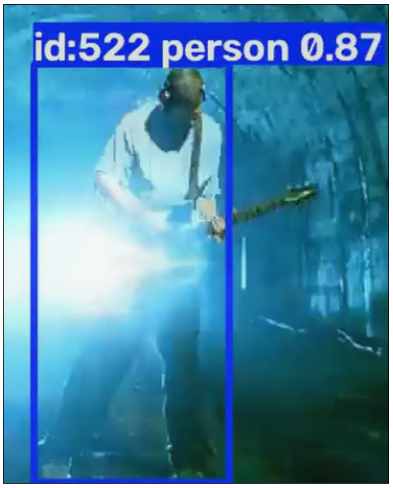
Ого тут людей. И всех видит. Особенно интересно ID=252 и 264.

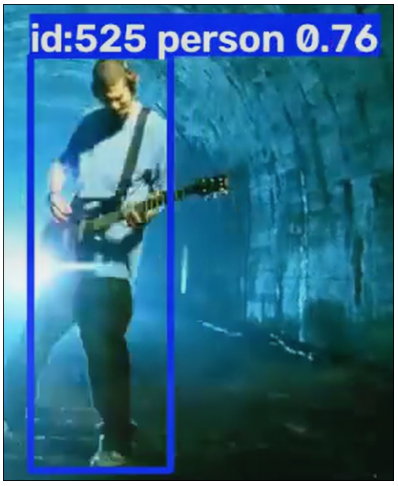
Опять же интересно, что из 9 человек, увидело только одного. Зато тень за человека приняло)))

Так же стоит отметить проблему частой смены ID, дальше в клипе будет момент, где у гитариста ID сменился 3 раза за 1 секунду. Также у разных людей в моментах остается один и тот же ID. Идеальным результатом было бы, если итоговое количество уникальных, выданных моделью ID, было бы равно количеству участников клипа, но это является довольно трудной задачей.

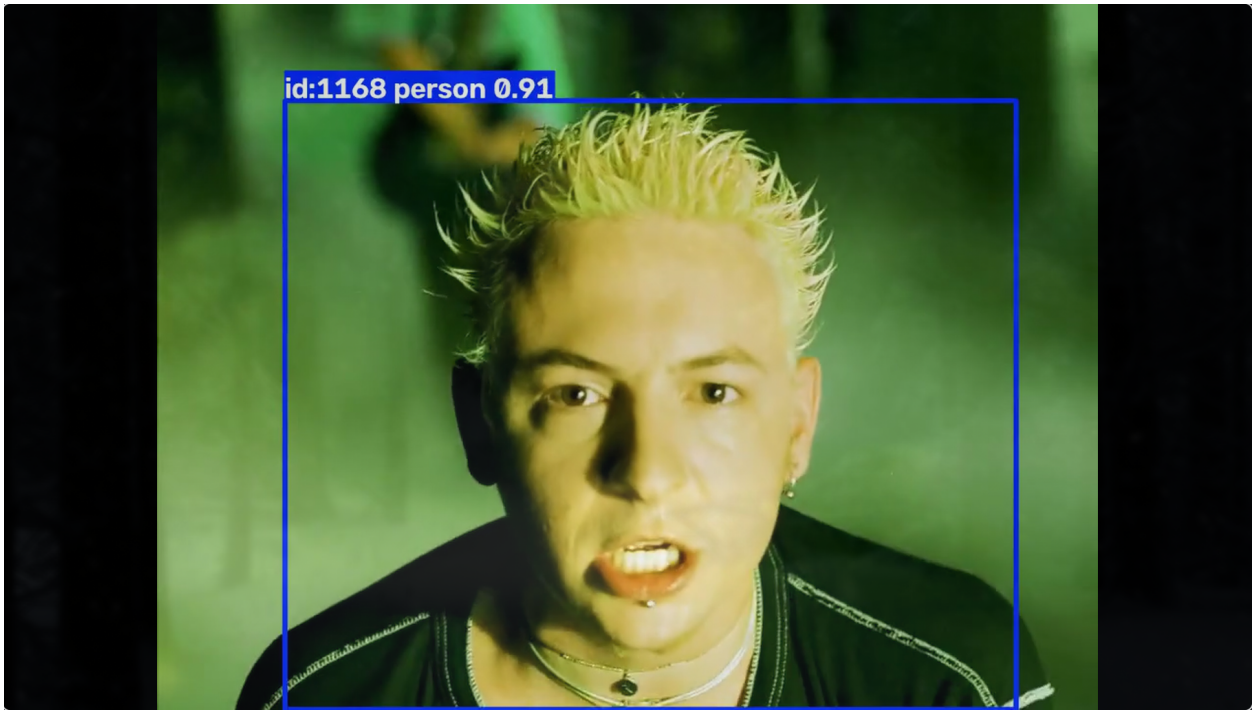








Забавные моменты с гитаристом(Брэд Дэльсон).



Последний кадр с милашкой Честером.

Часть 2:

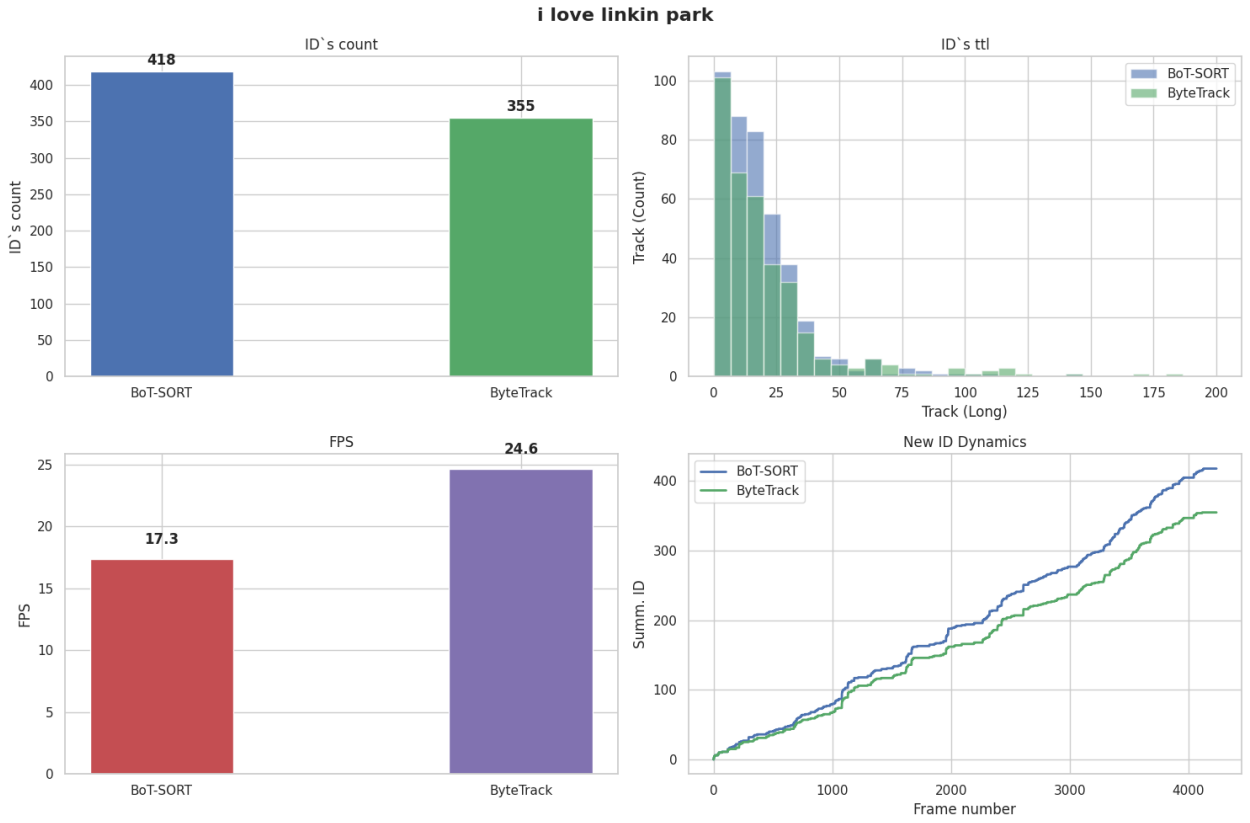
Теперь проведем сравнение предложенных алгоритмов — botsort и bytetrack. Пока что на их базовых настройках.

Код в main.py

Полученные метрики. Таблица результатов:

index	0	1
Tracker	botsort	bytetrack
Frames	4237	4237
FPS	17.33	24.6
Uniq ID	418	355
Avg. detections per frame	1.83	1.8
Avg. ID ttl	18.5	21.4
Median ttl ID	15	14
Max ttl ID	141	228
Short tracks (<10 frames)	145 (34.7%)	138 (38.9%)
Avg. confidence	0.699	0.711

Графики:



Сразу видно первенство ByteTrack. Он реже присваивал новые ID, работает быстрее и лучше. Из интересного, кривые на правом нижнем графике — имеют довольно сильное сходство. Это легко объясняется динамикой видео. Клип, в целом является хорошей идеей для данного исследования, ибо быстрые резко сменяющиеся кадры отлично годятся для стресс-теста алгоритмов.

Разберем основные параметры в yaml файлах:

Параметр	Описание	Базовые параметры BoT-SORT	Базовые параметры ByteTrack
tracker_type	Определяет алгоритм.	botsort	bytetrack
track_high_thresh	Порог уверенности для первой стадии сопоставления. Повышение очищает треки от ложных рамок, а понижение помогает удерживать объекты при смазывании кадра.	0.25	0.25
track_low_thresh	Порог для второй стадии сопоставления объектов с низкой уверенностью детектора. Позволяет подхватывать частично перекрытые цели.	0.1	0.1
new_track_thresh	Порог создания нового ID. Если объект не связался со старыми треками и его уверенность выше этого значения, создается новый ID. Повышение снижает спам дубликатами ID.	0.25	0.25
track_buffer	Время жизни потерянного трека. Увеличение помогает при длинных перекрытиях, но повышает риск ошибочного переключения ID.	30	30
match_thresh	Порог сходства пространственного перекрытия.	0.8	0.8
fuse_score	Объединение уверенности детектора со скоростью движения и IoU. Стабилизирует отслеживание при слабых детекциях	True	True
Только в BoT-SORT			
gmc_method	Компенсация движения камеры. Необходим для сцен с динамикой.	sparseOptFlow	-
proximity_thresh	Минимальный IoU для применения ReID. Повышение делает условия подключения внешних признаков более строгими.	0.5	-
appearance_thresh	Порог схожести внешнего вида (ReID embeddings). Повышение защищает от случайного обмена ID между похожими людьми.	0.8	-
with_reid	Включение модели повторной	False	-

	идентификации по внешнему виду. Требует дополнительных вычислительных ресурсов.		
model	Путь или имя модели ReID.	auto	-

Можно отметить, что параметры у двух алгоритмов абсолютно одинаковы, но ByteTrack все равно справился лучше, что говорит о его объективном преимуществе.

Из описания параметров, можно придти к следующим выводам: для достижения поставленной «цели»(снижение количества уникальных ID), необходимо внести следующие изменения:

- Увеличение track_buffer
- Увеличение new_track_thresh
- Уменьшение track_low_thresh

Проведем 3 эксперимента с разными параметрами. Значения представлены в таблице:

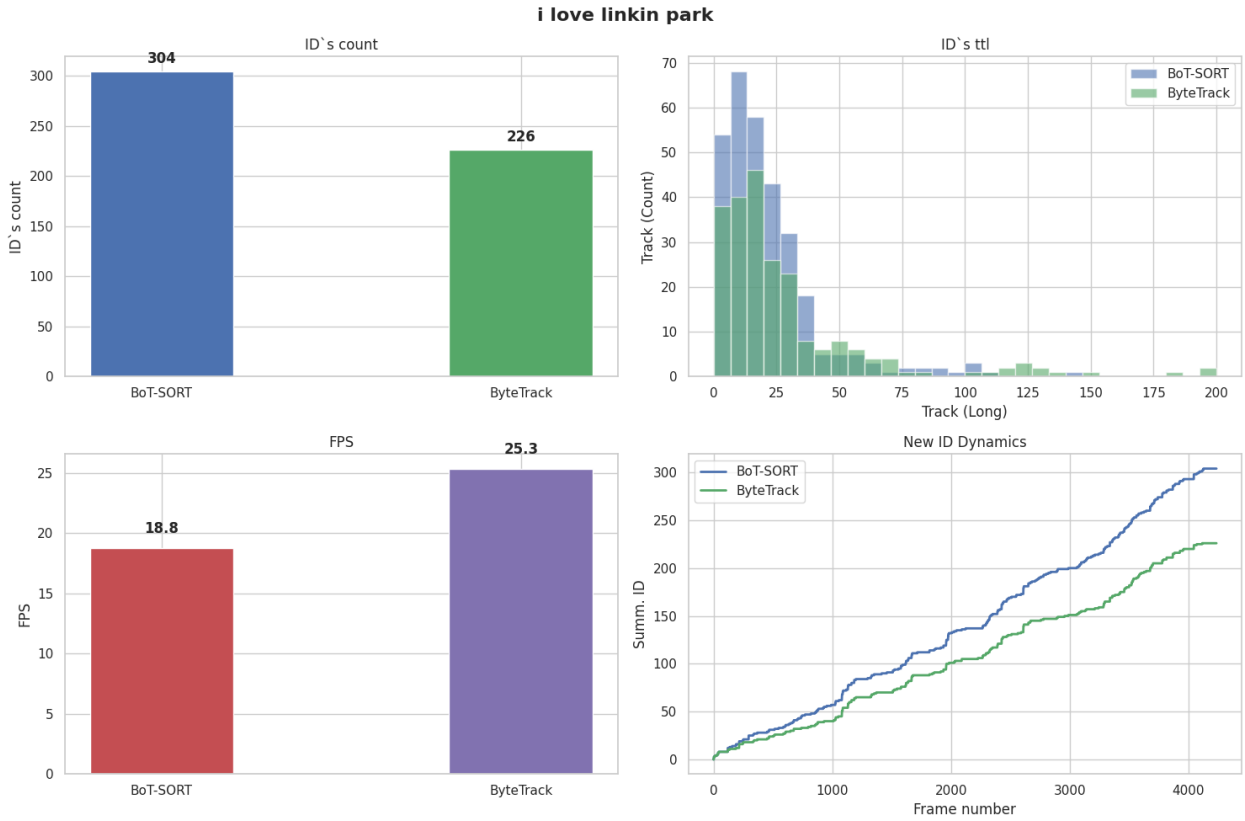
Эксперимент/ Значение	1	2	3
track_buffer	60	75	90
new_track_thresh	0.6	0.7	0.8
track_low_thresh	0.07	0.06	0.05

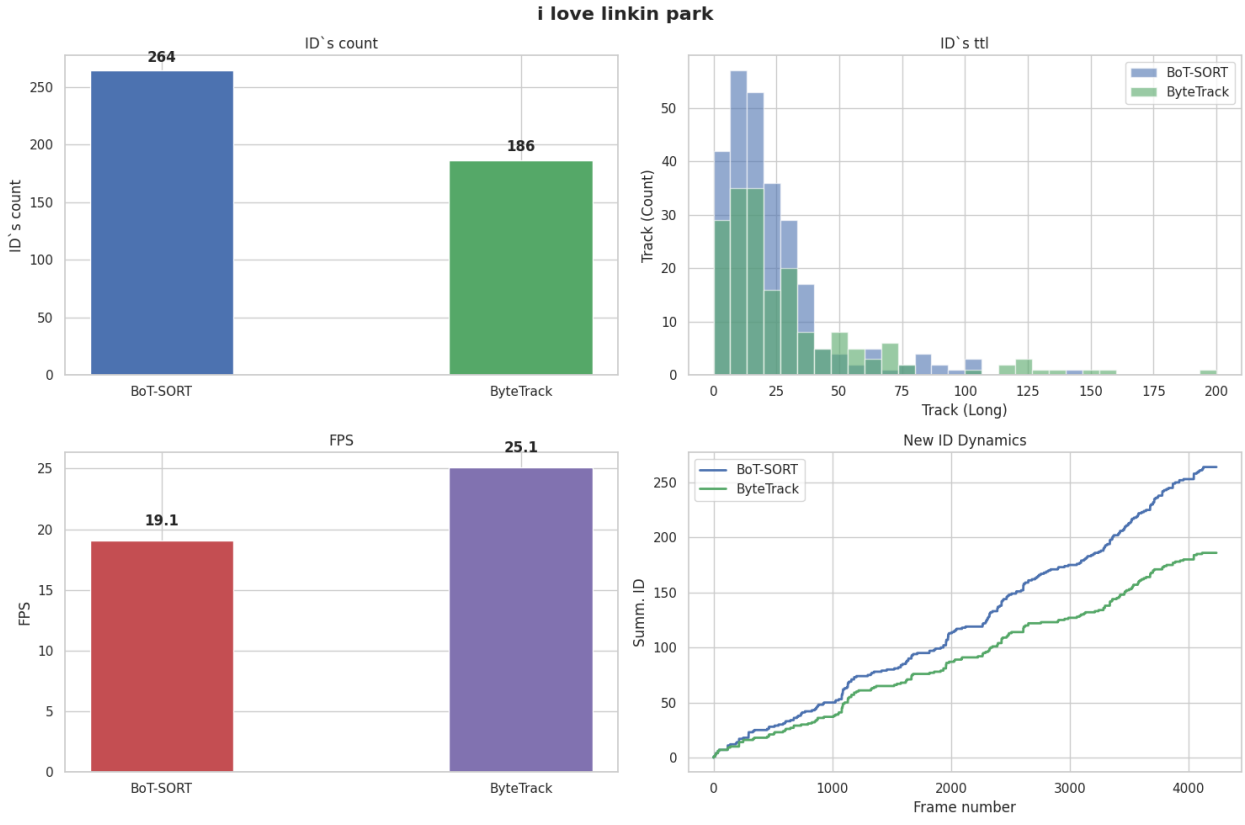
You're faking, you're mistaken
If you think that you could climb out of this hole
Forsaken, what would take me?
Analyzing by the power of your soul

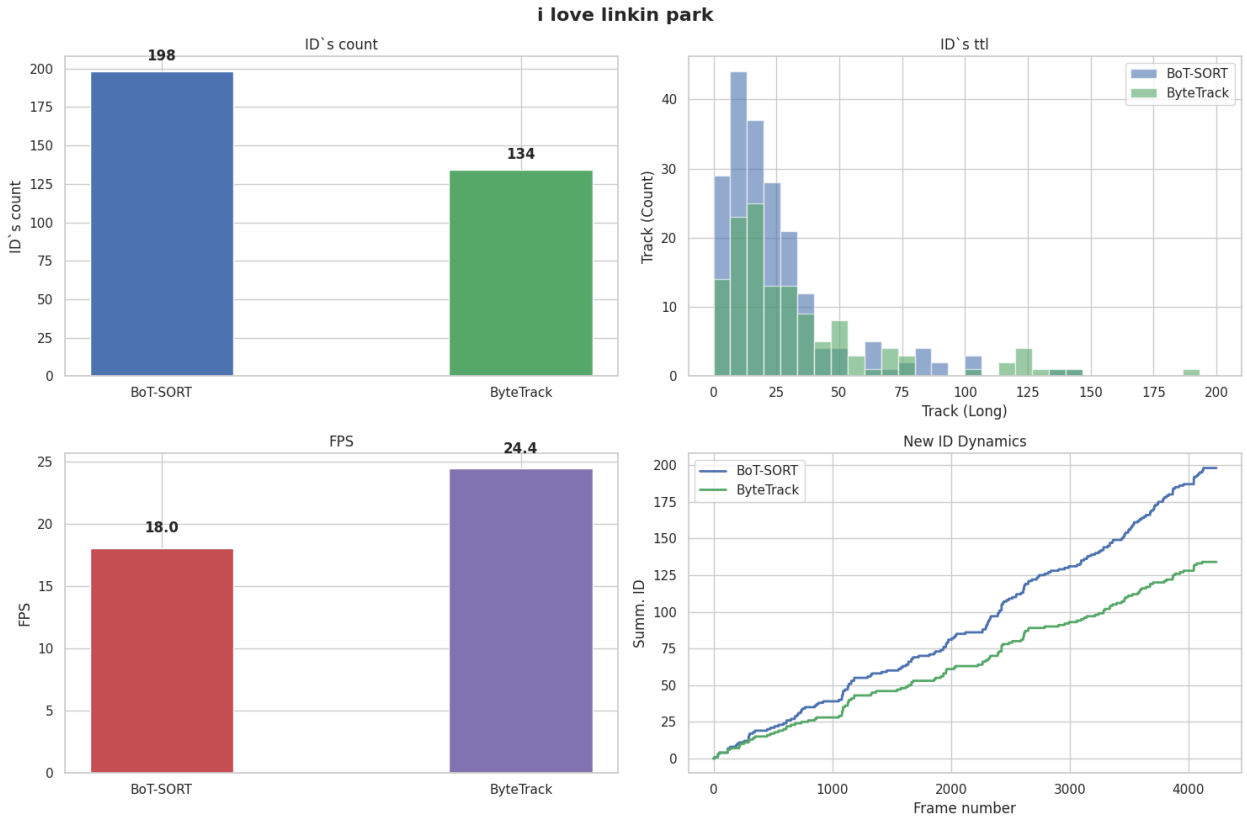
Broken down, a victim of your lies

You hide behind
Lies
You don't know
Why
You hide behind
Lies

Графики всех трех экспериментов:



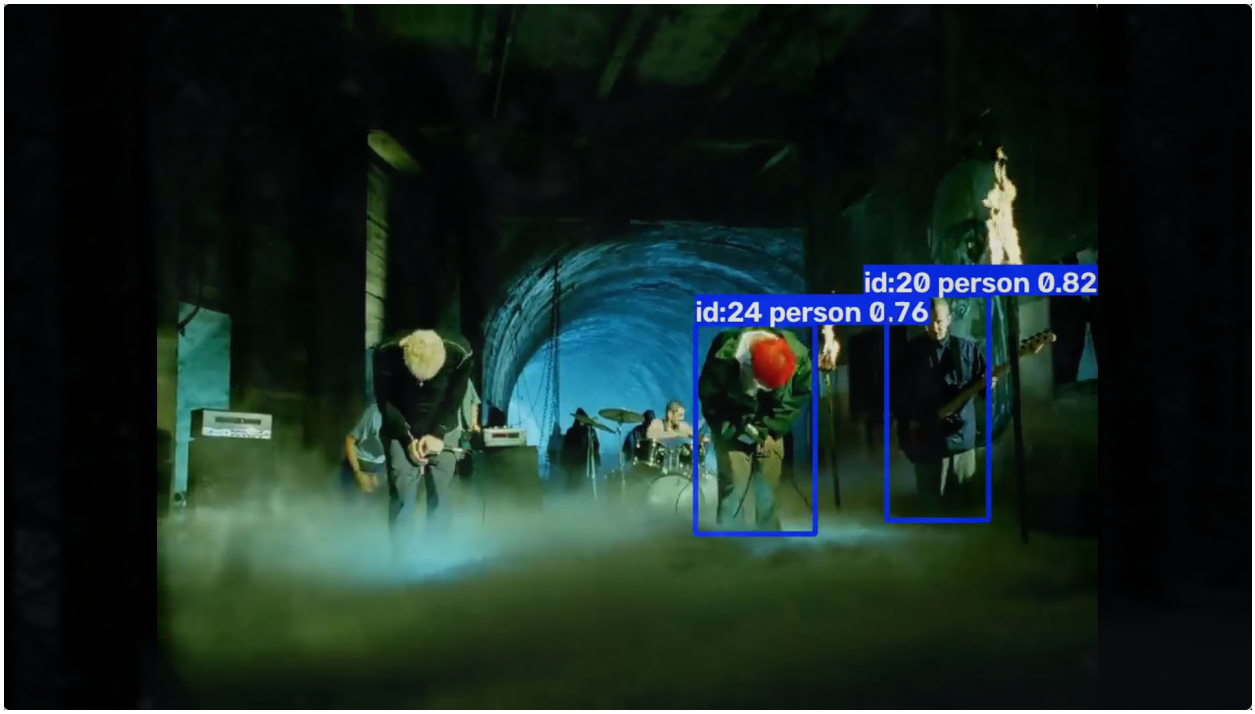




По графикам все выглядит прекрасно. Количество ID упало. Можно сделать вывод, что изменения выбранных параметров прямо влияет на качество(в положительном ключе). Однако для объективной оценки посмотрим проанализируем полученное видео. Посмотрим на примере последнего ByteTrack.

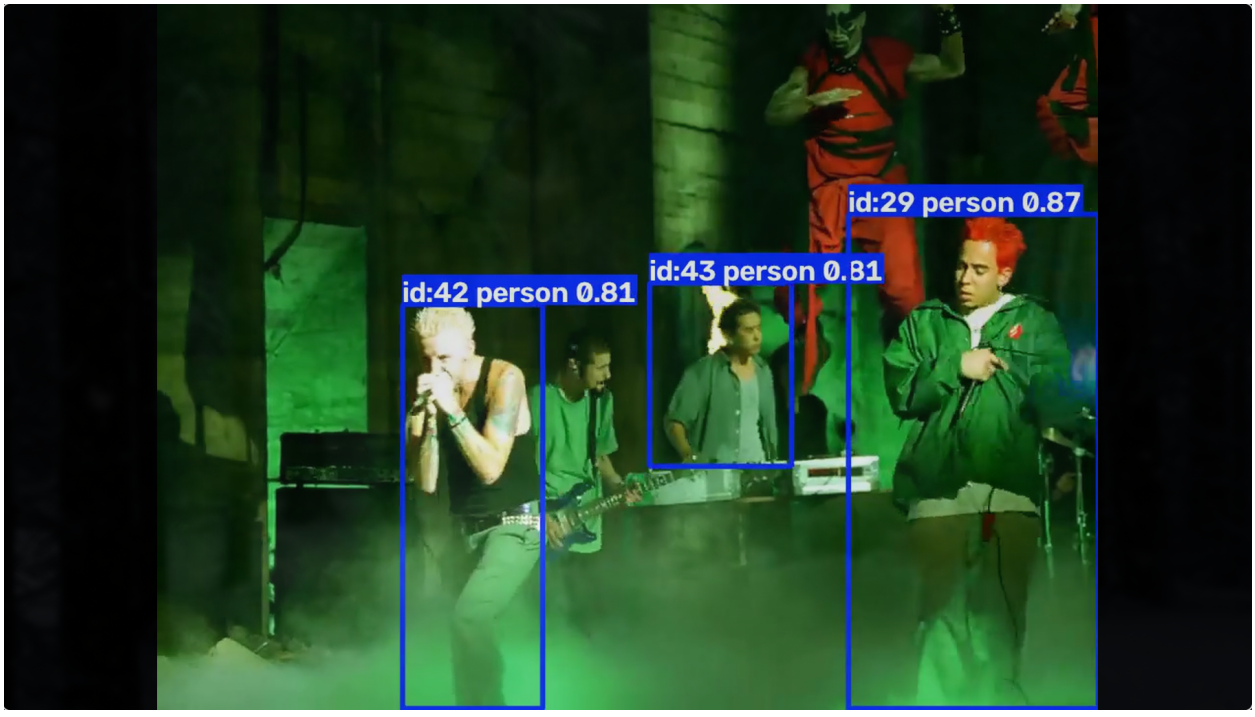
Мда… Сразу видим просевшее качество.

Зато не детектит тень)



Тут ситуация почти не изменилась. Разве что интересно, что модель решила затрекать Майка, но не Честера.

Огромная просадка качества тут. В прошлый раз модель определила всех людей в кадре.



Гитаристы не люди, понятно :(

Вообщем, качество трекинга заметно ухудшилось, модель присваивала один и тот же ID абсолютно разным людям(это проблема была и раньше, но в меньших масштабах).

Отсюда делается обратный нашему прошлому вывод: изменения параметров повлияло в негативном ключе. Модель стала более

осторожной и, видимо, предпочитала не трекать «сомнительного» человека, чтобы случайно не выдать ошибочный ID.

Увеличение окна в теории тоже оказалось слишком высоким. При пропажи человека из кадра, его ID удерживался в памяти ~3 секунд. Если в этой области кадра появлялся другой человек, то модель ошибочно подвязывал ему чужой ID. Для решения этой проблемы можно в теории подключить with_reid=True для проверки внешности.

Так что подбор параметров является неким trade-off. Исходные параметры лучше справлялись с детекцией. Да, они часто меняли ID, но тут вопрос, что лучше?

В идеале, конечно же, добиться оптимума между количественной детекцией и качественной детекцием, назовем это так. Очевидно, что, скорее всего, можно подобрать параметры, которые в нужной мере выполнят эту задачу. Но когда-нибудь потом.

Вывод: исследовал применение алгоритмов трекинга на базе обученной сети-детектора объектов. Линукс — лучшее ядро, Arch — лучший дистро, Linkin Park — лучшая группа, Я — использую arch btw.